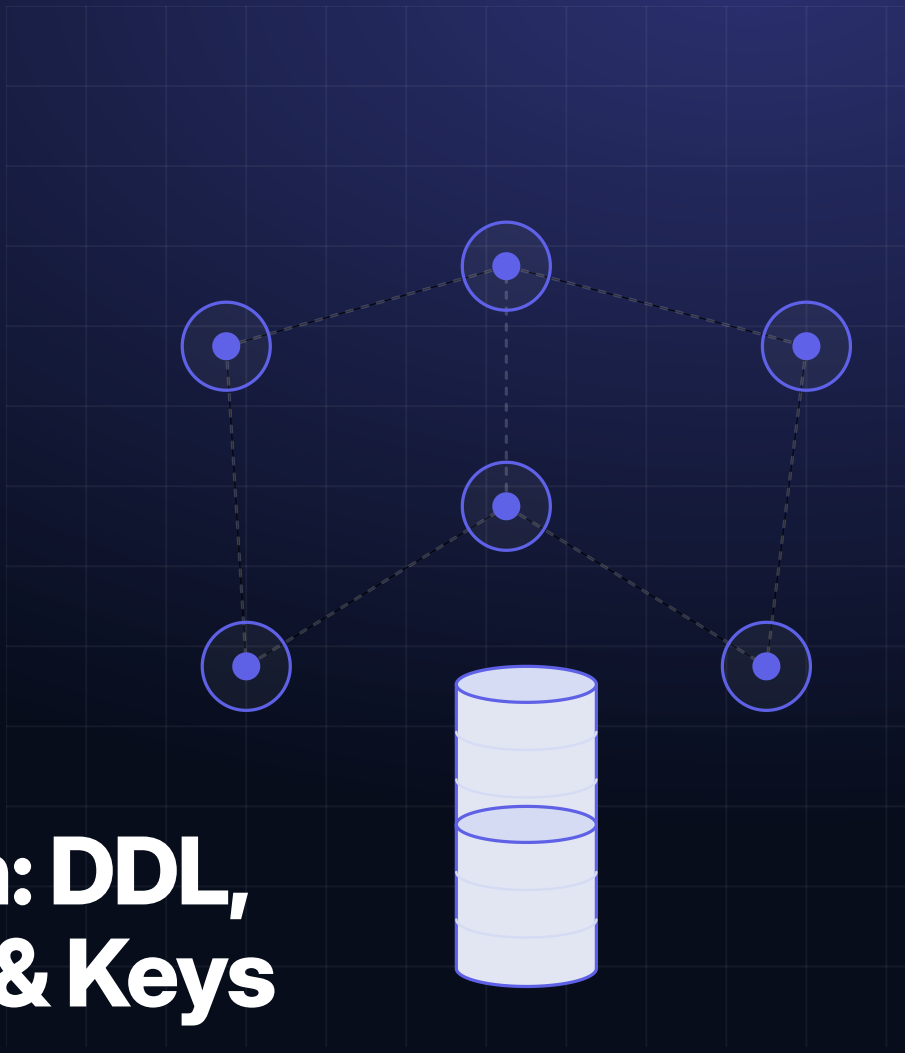


PHASE 02 / 07

Table Design: DDL, Constraints & Keys

Create, change and remove tables with DDL, protect your data with constraints, and learn the keys that connect a database together.

DDL Commands | Constraints | Keys in MySQL

Good pipelines start with good tables. This phase teaches you to design tables that reject bad data at the door and link cleanly to each other.

CONTENTS

Inside this phase

DDL Commands 3

- The four DDL commands and a table's life 3
- CREATE: databases and tables 4
- Choosing data types 5
- ALTER: change a table without losing its data 6
- DROP vs TRUNCATE vs DELETE 7
- Copying tables: LIKE and CTAS 8

Constraints 8

- Think of constraints as gates 10
- NOT NULL and UNIQUE 11
- DEFAULT, CHECK and AUTO_INCREMENT 12
- Column-level vs table-level, and naming 13

Keys in MySQL 14

- The key family tree 14
- PRIMARY KEY and UNIQUE KEY 15
- FOREIGN KEY: connecting tables 16
- COMPOSITE KEY: when one column is not enough 17
- SURROGATE vs NATURAL keys 18
- Normalization in one picture 20

Phase 2 Recap & Practice 21

- Cheat sheet 21
- Check yourself 21
- Practice lab 22
- Answer key 22

2.1 PART 2.1 · DATA DEFINITION LANGUAGE

DDL Commands

DDL commands define the *structure* of a database: its tables, columns and types. They do not work with individual rows; they work on the containers that hold them.

What you will learn

- Create databases and tables with `CREATE`
- Choose the right data type for every column
- Change a table with `ALTER` (add, rename, modify, drop)
- Know exactly how `DROP`, `TRUNCATE` and `DELETE` differ
- Copy tables with `CREATE TABLE … LIKE` and CTAS

Why it matters for data engineering. Every pipeline writes into tables someone had to design. Wrong types (money stored as FLOAT, dates stored as text) cause silent errors that surface months later in dashboards. Getting DDL right is cheap; fixing it after a billion rows is not.

The four DDL commands and a table's life

Your slides list four DDL commands: **CREATE**, **ALTER**, **DROP** and **TRUNCATE**. Here is what each does to a table's *structure* and its *rows*.

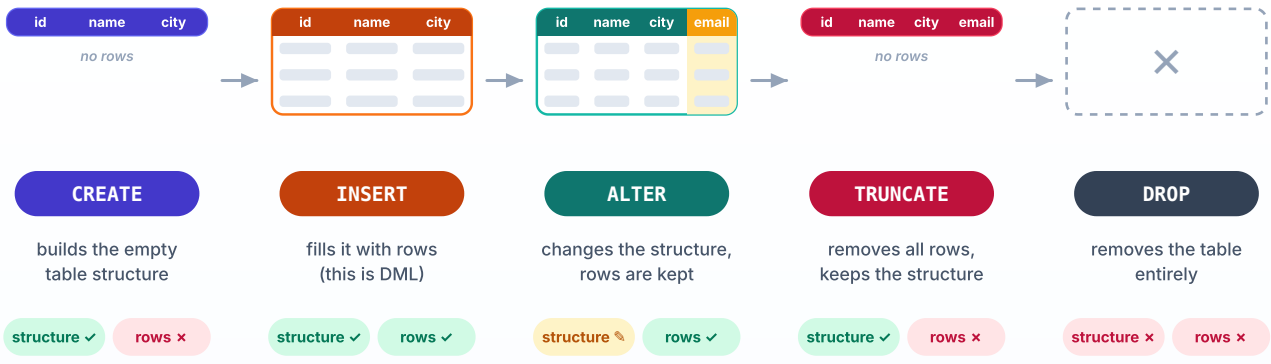


Figure 1. A table's life cycle. Only CREATE, ALTER, TRUNCATE and DROP are DDL; INSERT belongs to DML (Phase 4).

CREATE: databases and tables

Start with a database (a container, also called a *schema* in MySQL), then add tables to it.

SQL — Database-level DDL

```sql
CREATE DATABASE sales;          -- make the container
USE sales;                      -- make it the default for later statements
SHOW DATABASES;                 -- list all containers
DROP DATABASE IF EXISTS scratch; -- remove one (careful!)
```

A table is defined by listing each column with its **name**, **data type** and optional **constraints**. This is the first table from your course notes:

SQL — Course example: a minimal table

```sql
CREATE TABLE stores
(
    store_id   INT,
    store_name VARCHAR(200)
);
```

SQL — We gave a value for store_id only. What happens to store_name?

```sql
INSERT INTO stores (store_id) VALUES (3);
SELECT * FROM stores;
```

RESULT — 1 ROW

store_id	store_name
3	*NULL*

REMEMBER

Unlisted columns become **NULL** (meaning *unknown / missing*). Nothing in this table stops that. In the Constraints chapter you will see how `NOT NULL` closes this gap.

PRO TIP: WRITE RE-RUNNABLE SCRIPTS

Use `CREATE TABLE IF NOT EXISTS …` and `DROP TABLE IF EXISTS …` in scripts you may run more than once. This makes them **idempotent**: running twice gives the same result as running once, a core pipeline habit.

Choosing data types

A data type tells MySQL what kind of value a column holds and how much space it needs. The right type saves storage, speeds queries and rejects nonsense (you cannot store `'hello'` in an `INT`).

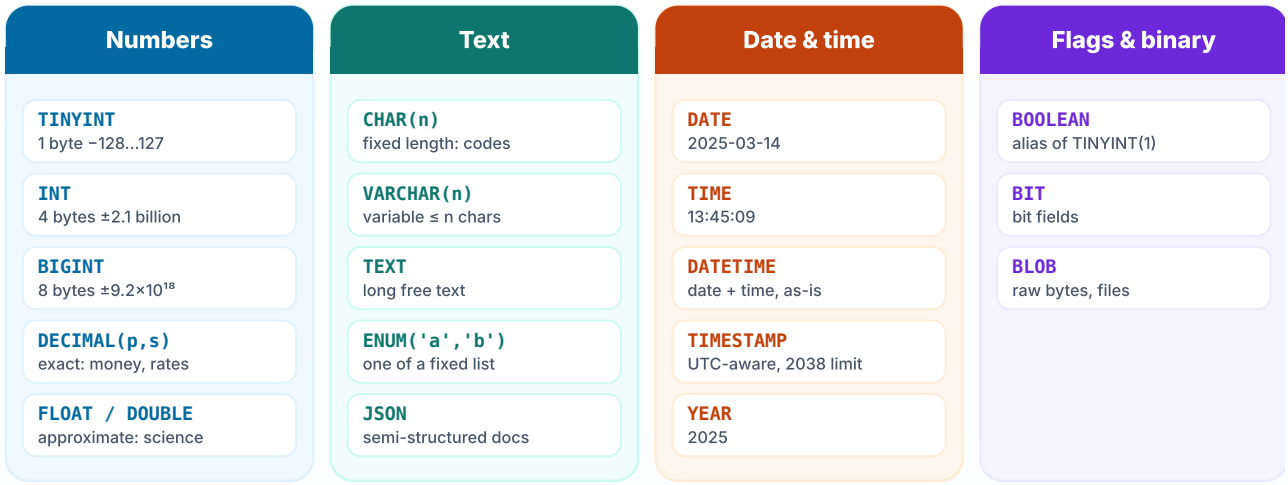


Figure 2. The four type families you will use daily. Inside each family the choice is a trade-off between range, precision and storage.

Question	Use	Avoid
Money, prices, rates?	`DECIMAL(10,2)` (exact)	`FLOAT` / `DOUBLE`: 0.1 + 0.2 ≠ 0.3 in binary
Surrogate IDs / counters?	`INT` (or `BIGINT` if > 2 billion rows)	`VARCHAR` for numeric IDs you will sort or join
Fixed-size codes (country, `'F'` / `'M'`)?	`CHAR(2)`, `CHAR(1)`	`TEXT`: unindexable length for tiny values
Names, emails, titles?	`VARCHAR(n)` with a sensible n	`VARCHAR(255)` everywhere by habit
A date with no time?	`DATE`	Text like `'14/03/2025'`: sorts wrongly, cannot do date math
An event instant?	`DATETIME` or `TIMESTAMP` (store UTC)	Local times with no timezone info
Yes/no flags?	`TINYINT(1)` / `BOOLEAN`	`'Y'` / `'N'` strings mixed with `1` / `0`

DATA-ENGINEERING LENS

`dim_product.unit_price` in your course database is `DECIMAL(10,2)`: up to 8 digits before the point, exactly 2 after. In a warehouse that is the difference between revenue that adds up to the cent and revenue that is off by a few paise on every report.

ALTER: change a table without losing its data

Requirements change: a new column is needed, a name was misspelled, a size was too small. `ALTER TABLE` edits the structure in place. First, a table to practise on, with the constraints you saw in the course (explained fully in the next chapter):

SQL — The course's second table

```sql
CREATE TABLE stores_new
(
    store_id   INT UNIQUE,
    store_name VARCHAR(200) NOT NULL
);
INSERT INTO stores_new VALUES (1, 'store_xyz');
SELECT * FROM stores_new;
```

RESULT — 1 ROW

store_id	store_name
1	store_xyz

Goal	Syntax
Add a column	`ALTER TABLE t ADD COLUMN c VARCHAR(100) [AFTER other];`
Rename a column	`ALTER TABLE t RENAME COLUMN old TO new;`
Change type / size / default	`ALTER TABLE t MODIFY COLUMN c VARCHAR(150) DEFAULT 'x';`
Rename + redefine in one go	`ALTER TABLE t CHANGE old new VARCHAR(150);`
Drop a column	`ALTER TABLE t DROP COLUMN c;`
Rename the table	`ALTER TABLE t RENAME TO t2;` or `RENAME TABLE t TO t2;`
Add / drop a constraint	`ALTER TABLE t ADD CONSTRAINT name … ;` `ALTER TABLE t DROP CONSTRAINT name;`

SQL — Add → rename → modify, then inspect the result

```sql
ALTER TABLE stores_new ADD COLUMN store_city VARCHAR(100) AFTER store_name;
ALTER TABLE stores_new RENAME COLUMN store_city TO store_location;
ALTER TABLE stores_new MODIFY COLUMN store_location VARCHAR(150) DEFAULT 'Unknown';
DESCRIBE stores_new;
```

RESULT — 3 ROWS

Field	Type	Null	Key	Default	Extra
store_id	int	YES	UNI	*NULL*	
store_name	varchar(200)	NO		*NULL*	
store_location	varchar(150)	YES		Unknown	

⚠ WATCH OUT

Your course script runs `RENAME COLUMN store_city TO store_location` on `stores_new`. That only works **after** a `store_city` column exists (we added it above); otherwise MySQL answers `ERROR 1054: Unknown column 'store_city'`.

DATA-ENGINEERING LENS

On big production tables, `ALTER` can lock the table or rewrite every row. Teams use online schema-change tools or add new columns as nullable first, backfill, then tighten. Never run a blind `ALTER` on a live billion-row table.

DROP vs TRUNCATE vs DELETE

These three are the most confused commands in SQL and a guaranteed interview question. They all remove something, but they remove different things.

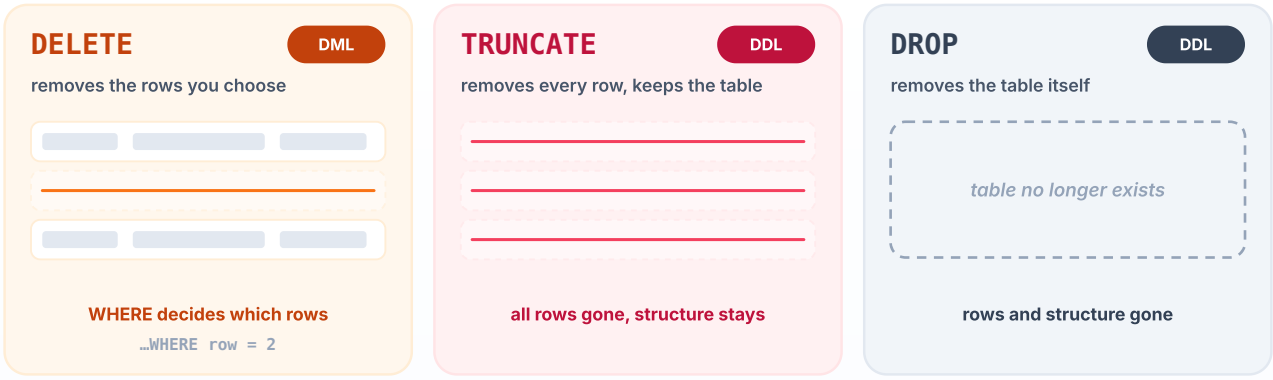


Figure 3. Dashed rows have been removed. DELETE is selective; TRUNCATE empties the table; DROP deletes the table.

	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	chosen rows (or all)	all rows	the whole table
`WHERE` **allowed?**	Yes	No	No
Table structure kept?	Yes	Yes	No
Rollback (undo)?	Yes, inside a transaction	No: auto-commits	No: auto-commits
`AUTO_INCREMENT` **counter**	Keeps counting	Resets to 1	Gone
Speed on big tables	Slow: row by row	Very fast	Very fast

Proof for the `AUTO_INCREMENT` row: we delete every row, insert a new one, then do the same with `TRUNCATE`.

SQL — After DELETE the counter keeps going: id = 4

```sql
DELETE FROM ai_demo;
INSERT INTO ai_demo (note) VALUES ('after delete');
SELECT * FROM ai_demo;
```

RESULT — 1 ROW

id	note
4	after delete

SQL — After TRUNCATE the counter restarts: id = 1

```sql
TRUNCATE TABLE ai_demo;
INSERT INTO ai_demo (note) VALUES ('after truncate');
SELECT * FROM ai_demo;
```

RESULT — 1 ROW

id	note
1	after truncate

INTERVIEW ANGLE

"How do you empty a 100-million-row staging table quickly?" `TRUNCATE` : it deallocates data pages instead of logging each row deletion. "...and if you only want last month's rows gone?" `DELETE … WHERE` (in batches).

Copying tables: LIKE and CTAS

SQL — CTAS creates a table from a query result (structure + data)

```sql
CREATE TABLE top_products AS                    -- CTAS: Create Table As Select
SELECT product_key, product_name, unit_price
FROM   sales.dim_product
ORDER  BY unit_price DESC
LIMIT  5;
SELECT * FROM top_products;
```

RESULT — 5 ROWS

product_key	product_name	unit_price
443	Likely Brother	999.04
466	Soon During	989.41
80	Car Report	986.45
228	Room Leg	986.00
171	Answer Hot	985.18

SQL — Structure first, then data

```sql
CREATE TABLE stores_backup LIKE stores_new;              -- same structure, no rows
INSERT INTO stores_backup SELECT * FROM stores_new;      -- now copy the rows
```

DATA-ENGINEERING LENS

CTAS is the building block of ELT. Load raw data into a staging table, then `CREATE TABLE clean_x AS SELECT … FROM raw_x` to materialize each transformation step. Note: CTAS copies column types but **not** keys, indexes or constraints. Add them afterwards with `ALTER TABLE`.

2.2 PART 2.2 · DATA INTEGRITY

Constraints

Constraints are rules attached to columns or tables. The database itself enforces them on every INSERT and UPDATE, no matter which application, script or person sends the data.

What you will learn

- Apply `NOT NULL`, `UNIQUE`, `DEFAULT`, `CHECK` and `AUTO_INCREMENT`
- Read the error messages MySQL raises when a rule is broken
- Name, add and drop constraints on existing tables
- Decide where strict rules belong in a data pipeline

Why it matters for data engineering. Bad data is cheaper to stop at the door than to clean later. A constraint is a free, always-on data-quality test that runs on every write. Many cloud warehouses do not enforce them, which is exactly why you need to understand what they *would* guarantee.

Think of constraints as gates

Every incoming row walks through the gates below. If any gate says no, the **whole statement is rejected** and nothing is stored. That all-or-nothing behavior keeps tables trustworthy.

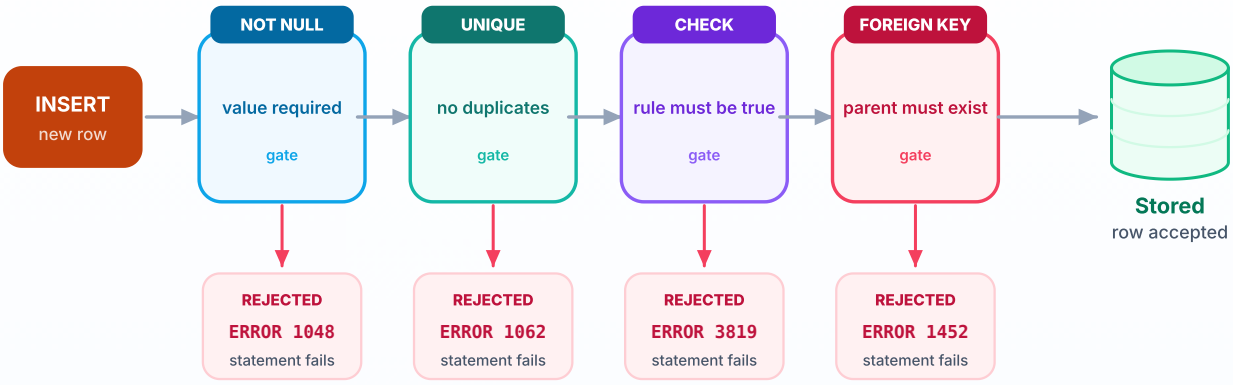


Figure 4. A row must pass every gate to be stored. PRIMARY KEY (next chapter) combines NOT NULL + UNIQUE.

NOT NULL and UNIQUE

NOTE

Your slides say UNIQUE "can have one NULL (in most databases)". **In MySQL a UNIQUE column can hold many NULLs**, because two unknowns are never considered equal. SQL Server allows only one. It is a classic cross-database trap.

SQL — NOT NULL blocks a missing name

```sql
INSERT INTO stores_new VALUES (2, NULL);
```

ERROR

```
ERROR 1048 (23000): Column 'store_name' cannot be null
```

SQL — UNIQUE blocks a second store_id = 1

```sql
INSERT INTO stores_new VALUES (1, 'another store');
```

ERROR

```
ERROR 1062 (23000): Duplicate entry '1' for key 'stores_new.store_id'
```

SQL — Two NULLs in a UNIQUE column are allowed in MySQL

```sql
INSERT INTO stores_new (store_id, store_name) VALUES (NULL, 'no id yet'), (NULL, 'also no id');
SELECT store_id, store_name FROM stores_new ORDER BY store_name;
```

RESULT — 3 ROWS

store_id	store_name
NULL	also no id
NULL	no id yet
1	store_xyz

DEFAULT, CHECK and AUTO_INCREMENT

These three make tables friendlier: fill in missing values, enforce business rules, and number rows automatically.

SQL — Constraints defined at the bottom are called table-level constraints

```sql
CREATE TABLE products_demo
(
    product_id   INT AUTO_INCREMENT PRIMARY KEY,            -- numbers itself: 1, 2, 3 …
    sku          VARCHAR(20)  NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    unit_price   DECIMAL(10,2) NOT NULL,
    status       VARCHAR(10)  NOT NULL DEFAULT 'active',    -- used when omitted
    created_at   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,     -- stamp every load
    CONSTRAINT uq_sku    UNIQUE (sku),
    CONSTRAINT chk_price CHECK (unit_price > 0)
);
INSERT INTO products_demo (sku, product_name, unit_price)
VALUES ('SKU-1', 'Notebook', 49.90), ('SKU-2', 'Pen', 9.50);
SELECT product_id, sku, product_name, unit_price, status FROM products_demo;
```

RESULT — 2 ROWS

product_id	sku	product_name	unit_price	status
1	SKU-1	Notebook	49.90	active
2	SKU-2	Pen	9.50	active

SQL — CHECK rejects a negative price

```sql
INSERT INTO products_demo (sku, product_name, unit_price) VALUES ('SKU-3', 'Bad pen', -5);
```

ERROR

```
ERROR 3819 (HY000): Check constraint 'chk_price' is violated.
```

SQL — Naming a constraint (uq_sku) makes the error self-explanatory

```sql
INSERT INTO products_demo (sku, product_name, unit_price) VALUES ('SKU-1', 'Clone', 12);
```

ERROR

```
ERROR 1062 (23000): Duplicate entry 'SKU-1' for key 'products_demo.uq_sku'
```

MYSQL SPECIFIC

`CHECK` constraints are enforced from **MySQL 8.0.16** onward. Older MySQL parsed them but silently ignored them. Always check your version before relying on `CHECK`.

Column-level vs table-level, and naming

SQL — Column-level

```sql
-- column-level: next to the column
unit_price DECIMAL(10,2)
    NOT NULL CHECK (unit_price > 0),
sku VARCHAR(20) UNIQUE
```

SQL — Table-level (named)

```sql
-- table-level: after all columns
CONSTRAINT chk_price
    CHECK (unit_price > 0),
CONSTRAINT uq_sku UNIQUE (sku)
```

- Use **table-level** syntax when the rule spans several columns, e.g. `UNIQUE (store_id, product_id)`.
- **Name your constraints** (`uq_`, `chk_`, `fk_`, `pk_` prefixes). Error messages and `ALTER … DROP CONSTRAINT` then become readable.
- Constraints can be added later: `ALTER TABLE t ADD CONSTRAINT chk_qty CHECK (qty >= 0);`. MySQL validates all existing rows first, so dirty data blocks the change.

Constraint	Rule it enforces	NULLs?	Per table
`NOT NULL`	A value must be supplied	Never	Any number of columns
`UNIQUE`	No two rows share the value(s)	Many NULLs allowed	Any number
`PRIMARY KEY`	Unique + not null; the row's identity	Never	Exactly one
`FOREIGN KEY`	Value must exist in the parent table	NULL allowed (= no parent)	Any number
`CHECK`	A boolean expression must be true	NULL passes the check	Any number
`DEFAULT`	Fill a value when none is given	n/a (not a rule)	One per column

DATA-ENGINEERING LENS

Where should constraints live in a pipeline? *Staging/raw layer*: keep them loose so a bad row does not stop an entire load (you want to capture and quarantine it). *Modeled/warehouse layer*: be strict. Note that Snowflake, BigQuery and Redshift **do not enforce** `PRIMARY KEY` / `UNIQUE` / `FOREIGN KEY`; they are documentation only, so you must test uniqueness yourself with queries or dbt tests.

2.3 PART 2.3 · IDENTITY AND RELATIONSHIPS

Keys in MySQL

Keys answer two questions every database must solve: *how do I point at exactly one row?* and *how do rows in different tables belong together?*

What you will learn

- Define super, candidate, primary, alternate (unique) keys
- Link tables with foreign keys and choose referential actions
- Use composite keys for many-to-many tables
- Tell surrogate keys from natural keys, and why warehouses prefer surrogates
- Spot normalized vs flat designs

Why it matters for data engineering. Joins (Phase 4) are only as reliable as the keys behind them. Duplicate or missing keys are the number one cause of inflated revenue numbers and mysteriously dropped rows in pipelines.

The key family tree

Take a table `employees(emp_id, email, phone, full_name)`. Any set of columns that can identify a row is a **super key**. The *smallest* such sets are **candidate keys**. You pick one candidate as the **primary key**; the rest become **alternate (unique) keys**.

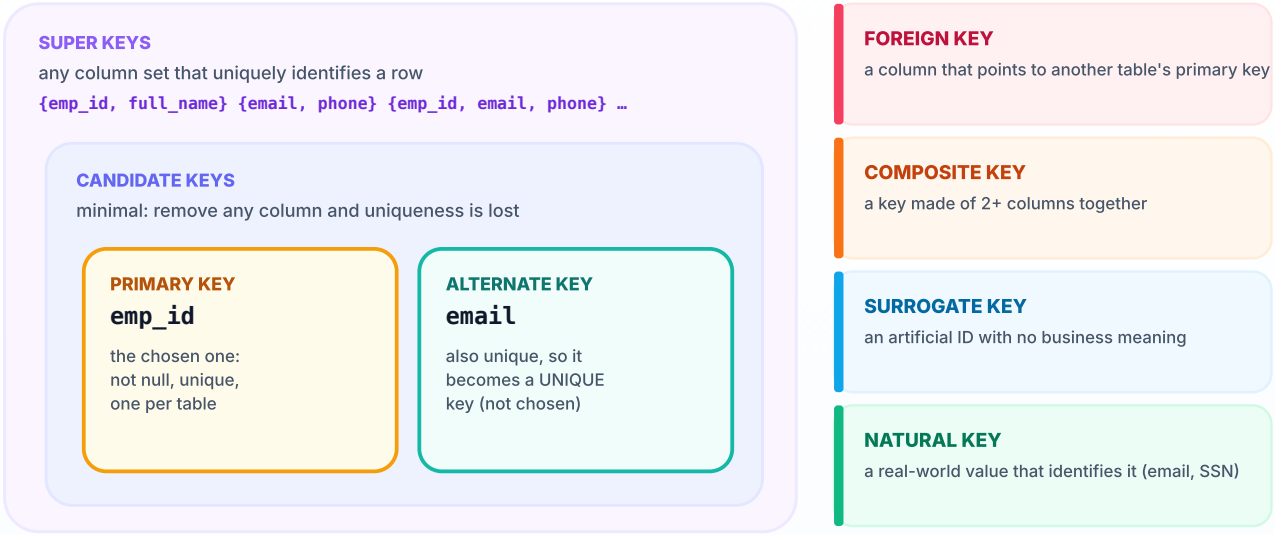


Figure 5. Left: how super, candidate, primary and alternate keys nest. Right: the other key terms from your course slides.

PRIMARY KEY and UNIQUE KEY

	PRIMARY KEY	UNIQUE KEY
Purpose	The row's identity	Prevent duplicates in another column
Per table	Exactly **one**	As many as you need
NULL allowed?	Never	Yes (many, in MySQL)
Index created?	Yes: the *clustered* index in InnoDB	Yes: a secondary unique index
Referenced by a foreign key?	Typically	Possible, but unusual

SQL — A parent table, a child table, and all three key types

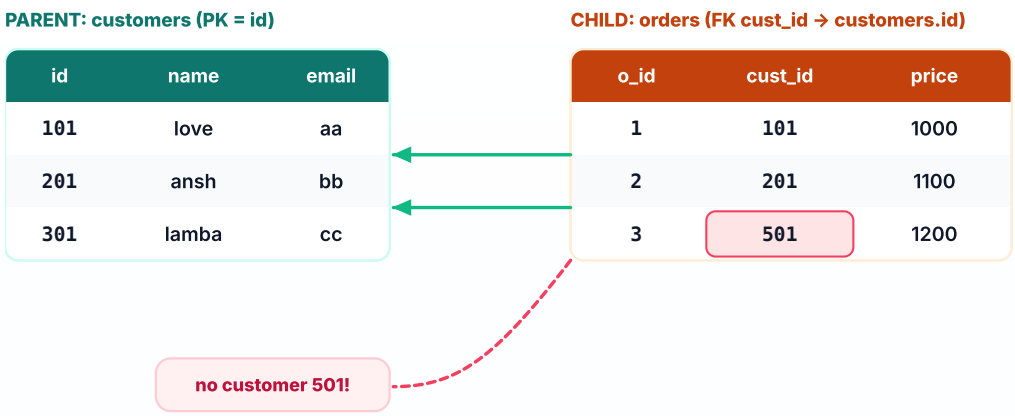
```sql
CREATE TABLE departments
(
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
CREATE TABLE employees
(
    emp_id    INT AUTO_INCREMENT PRIMARY KEY,
    email     VARCHAR(100) NOT NULL UNIQUE,        -- alternate key
    full_name VARCHAR(100) NOT NULL,
    dept_id   INT,
    CONSTRAINT fk_emp_dept FOREIGN KEY (dept_id)
        REFERENCES departments (dept_id)
        ON DELETE SET NULL ON UPDATE CASCADE
);
INSERT INTO departments VALUES (10, 'Data'), (20, 'Sales');
INSERT INTO employees (email, full_name, dept_id) VALUES
 ('asha@x.com', 'Asha', 10), ('ravi@x.com', 'Ravi', 10), ('meena@x.com', 'Meena', 20);
SELECT * FROM employees;
```

RESULT — 3 ROWS

emp_id	email	full_name	dept_id
1	asha@x.com	Asha	10
2	ravi@x.com	Ravi	10
3	meena@x.com	Meena	20

FOREIGN KEY: connecting tables

A foreign key makes a column in the **child** table point to the primary key of the **parent** table. MySQL then guarantees that no child row is an *orphan*, a row pointing to a parent that does not exist.



✓ orders 1 and 2 have a parent. × order 3 is an orphan; a foreign key would reject it (ERROR 1452).

Figure 6. This is the course's `ecom` data. Without a foreign key, nothing stops order 3 from pointing to a customer who does not exist.

SQL — Department 99 does not exist → insert rejected

```sql
INSERT INTO employees (email, full_name, dept_id) VALUES ('new@x.com', 'Nia', 99);
```

```
⚠ ERROR
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`lab`.`employees`, CONSTRAINT `fk_emp_dept` FOREIGN KEY (`dept_id`) REFERENCES `departments`
(`dept_id`) ON DELETE SET NULL ON UPDATE CASCADE)
```

Referential actions: what happens when a parent row is deleted



Figure 7. The ON DELETE clause of a foreign key decides what the database does with child rows. ON UPDATE works the same way for key changes.

SQL — Meena's department is gone, so her dept_id became NULL

```sql
DELETE FROM departments WHERE dept_id = 20;       -- fk_emp_dept says ON DELETE SET NULL
SELECT emp_id, full_name, dept_id FROM employees ORDER BY emp_id;
```

RESULT — 3 ROWS

emp_id	full_name	dept_id
1	Asha	10
2	Ravi	10
3	Meena	NULL

SQL — With no ON DELETE clause (RESTRICT) the parent cannot be deleted

```sql
CREATE TABLE d2 (id INT PRIMARY KEY);
CREATE TABLE e2 (id INT PRIMARY KEY, d INT, CONSTRAINT fk_e2_d2 FOREIGN KEY (d) REFERENCES d2 (id));
INSERT INTO d2 VALUES (1); INSERT INTO e2 VALUES (1, 1);
DELETE FROM d2 WHERE id = 1;
```

ERROR

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`lab`.`e2`,
CONSTRAINT `fk_e2_d2` FOREIGN KEY (`d`) REFERENCES `d2` (`id`))
```

WATCH OUT

Order matters. To create tables you must create the **parent first**; to drop them, drop the **child first**. That is why your course script drops `fact_sales` (child) before the `dim_` tables (parents).

COMPOSITE KEY: when one column is not enough

In an order-lines table, a product can appear in many orders and an order can contain many products. Neither `order_id` nor `product_id` is unique alone, but the *pair* is. A primary key made of several columns is a **composite key**; this is the standard design for many-to-many link tables.

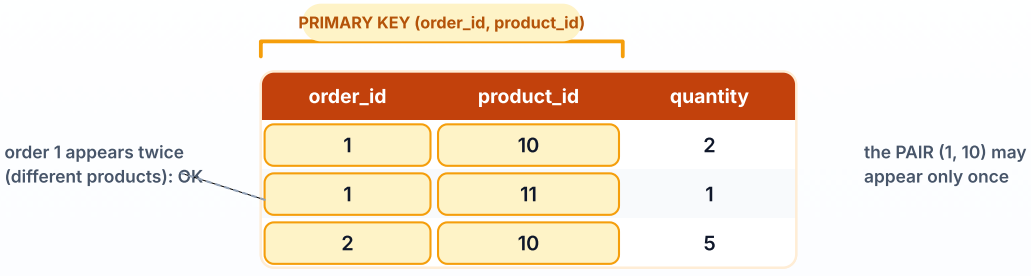


Figure 8. Individually the key columns repeat; together they never do.

SQL

```sql
CREATE TABLE order_items
(
    order_id   INT,
    product_id INT,
    quantity   INT NOT NULL,
    PRIMARY KEY (order_id, product_id)
);
INSERT INTO order_items VALUES (1, 10, 2), (1, 11, 1), (2, 10, 5);
SELECT * FROM order_items;
```

RESULT — 3 ROWS

order_id	product_id	quantity
1	10	2
1	11	1
2	10	5

SQL — The pair (1, 10) already exists

```sql
INSERT INTO order_items VALUES (1, 10, 9);
```

⚠ ERROR

```
ERROR 1062 (23000): Duplicate entry '1-10' for key 'order_items.PRIMARY'
```

SURROGATE vs NATURAL keys

A **natural key** is a real-world value (an email, a passport number, `CUST0001`). A **surrogate key** is an artificial number (`1, 2, 3 …`) that exists only to identify the row. Your own `dim_customer` table has both:

SQL — customer_key = surrogate key (PK) · customer_id = natural / business key

```sql
SELECT customer_key, customer_id, first_name, last_name FROM dim_customer ORDER BY customer_key
LIMIT 5;
```

RESULT — 5 ROWS

customer_key	customer_id	first_name	last_name
1	CUST0001	Antonio	Kelly
2	CUST0002	Todd	Lucas
3	CUST0003	Matthew	Lee
4	CUST0004	Gregory	Young
5	CUST0005	Nathan	Smith

	Surrogate key	Natural key
Example	`customer_key = 17`	`customer_id = 'CUST0017'`, `email`

	Surrogate key	Natural key
Meaning	None: just an ID	Real-world meaning
Stability	Never changes	Can change (email edit, re-numbering)
Size / join speed	Small `INT`: fast joins	Longer strings: slower, bigger indexes
Source-system independence	Yes: survives source migrations	Tied to the source system
Slowly changing history (SCD2)	Each version gets a new key	Same value repeats across versions

Illustration: one real customer, two historical versions (SCD Type 2)

customer_key	customer_id	city	valid_from	valid_to	is_current
17	CUST0017	Pune	2022-01-01	2024-06-30	0
1017	CUST0017	Mumbai	2024-07-01	9999-12-31	1

The natural key CUST0017 repeats; the surrogate key keeps each version unique, so old facts still point to the city the customer lived in at the time of the sale.

Figure 9. Why warehouses add surrogate keys: they let you keep history without breaking uniqueness.

DATA-ENGINEERING LENS

Profile before you trust a key. Before using a column as a key, prove it is unique and complete. This is what a candidate-key check looks like:

SQL — A column is a candidate key if distinct count = total rows and it has no NULLs

```sql
SELECT COUNT(*)                    AS total_rows,
       COUNT(DISTINCT customer_key)  AS distinct_key,
       COUNT(DISTINCT customer_id)   AS distinct_cust_id,
       COUNT(DISTINCT email)         AS distinct_email,
       COUNT(DISTINCT country)       AS distinct_country
FROM   dim_customer;
```

RESULT — 1 ROW

total_rows	distinct_key	distinct_cust_id	distinct_email	distinct_country
500	500	500	499	207

Normalization in one picture

Storing everything in one wide table repeats the same facts again and again. **Normalization** splits such a table so each fact lives in exactly one place, tied together with keys. This is how OLTP systems like MySQL are designed.

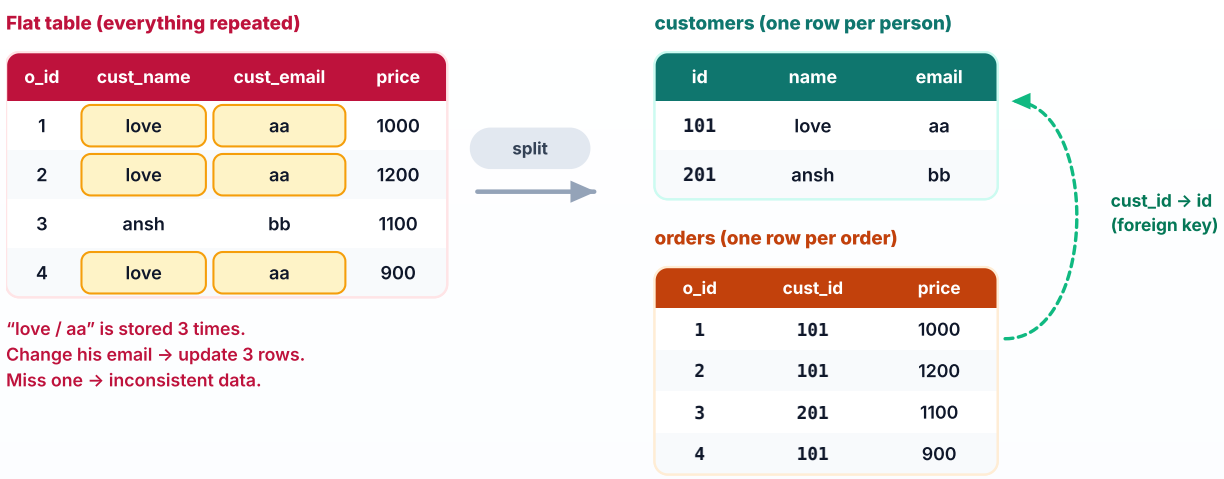


Figure 10. After normalizing, each customer is stored once and orders point to them. Joins (Phase 4) reassemble the wide view when you need it.

Normal form	Rule in plain words	Fixes
1NF	One value per cell; no repeating groups (no `'a,b,c'` lists in a column)	Unsearchable packed values
2NF	Every non-key column depends on the **whole** key (matters for composite keys)	Partial dependencies
3NF	Non-key columns depend on the key and **nothing else**	Repeated facts like city→state

DATA-ENGINEERING LENS

Normalize for OLTP, denormalize for OLAP. Your `sales` schema is *deliberately* denormalized into dimensions (a star), trading some repetition for far simpler and faster analytical queries. Know both; choose by workload.

INTERVIEW ANGLE

"Primary key vs unique key?" Primary: one per table, never NULL, defines identity. Unique: many per table, allows NULLs, prevents duplicates. "What is a foreign key for?" Referential integrity: no orphan rows.

2.4 WRAP-UP

Phase 2 Recap & Practice

Check your grasp of DDL, constraints and keys, then design a small schema of your own.

Cheat sheet

Topic	Remember
DDL	`CREATE` , `ALTER` , `DROP` , `TRUNCATE` change structure; they auto-commit
Data types	`DECIMAL` for money · `INT` / `BIGINT` for IDs · `VARCHAR(n)` text · `DATE` / `DATETIME` time
ALTER	`ADD COLUMN` · `RENAME COLUMN a TO b` · `MODIFY COLUMN` · `DROP COLUMN` · `RENAME TO`
Remove data	`DELETE` rows (undoable) · `TRUNCATE` all rows (fast, resets counter) · `DROP` the table
Copy tables	`CREATE TABLE t2 LIKE t` (structure) · `CREATE TABLE t2 AS SELECT …` (CTAS)
Constraints	`NOT NULL` · `UNIQUE` · `DEFAULT` · `CHECK` · `PRIMARY KEY` · `FOREIGN KEY` · `AUTO_INCREMENT`
Keys	Super ⊃ Candidate → Primary + Alternate · Foreign · Composite · Surrogate vs Natural
FK actions	`ON DELETE RESTRICT | CASCADE | SET NULL` ; create parents first, drop children first
Design	Normalize OLTP (3NF) · star-schema OLAP · surrogate keys in dimensions

Check yourself

Q1 What is the difference between `DROP` , `TRUNCATE` and `DELETE` ?

DELETE removes chosen rows (DML, rollback-able). TRUNCATE removes all rows, keeps structure, resets AUTO_INCREMENT, cannot be rolled back. DROP removes the table itself.

Q2 Why store prices as `DECIMAL` and not `FLOAT` ?

FLOAT is binary-approximate and can produce rounding errors (0.1 + 0.2 ≠ 0.3); DECIMAL is exact.

Q3 Can a table have two primary keys? Two unique keys?

One primary key (possibly composite); any number of unique keys.

Q4 What is an orphan row and which constraint prevents it?

A child row whose foreign key value has no matching parent; a FOREIGN KEY constraint.

Q5 Why does a warehouse prefer surrogate keys on dimensions?

Stable small integers, independent of source systems, and they allow history (SCD Type 2) without breaking uniqueness.

Practice lab

1. Create `students(student_id INT auto-increment PK, email unique & not null, name not null, age with a CHECK between 16 and 100, enrolled_on DATE default today)`.
2. Add a column `phone VARCHAR(15)`, then rename it to `mobile`.
3. Create `enrollments(student_id, course_code, grade)` with a **composite primary key** and a foreign key to `students` that **cascades** deletes.
4. Insert two students and two enrollments, delete one student, and confirm that the enrollments disappeared.
5. Make a backup copy of `students` with CTAS, then try to `TRUNCATE` the original. What happens, and why?

Answer key

SQL — Solutions 1 to 5a (executed against a live server)

```sql
CREATE TABLE students (
    student_id  INT AUTO_INCREMENT PRIMARY KEY,
    email       VARCHAR(100) NOT NULL UNIQUE,
    name        VARCHAR(100) NOT NULL,
    age         INT,
    enrolled_on DATE DEFAULT (CURRENT_DATE),
    CONSTRAINT chk_age CHECK (age BETWEEN 16 AND 100)
);
ALTER TABLE students ADD COLUMN phone VARCHAR(15);
ALTER TABLE students RENAME COLUMN phone TO mobile;

CREATE TABLE enrollments (
    student_id  INT,
    course_code VARCHAR(10),
    grade       CHAR(1),
    PRIMARY KEY (student_id, course_code),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
        REFERENCES students (student_id) ON DELETE CASCADE
);
INSERT INTO students (email, name, age) VALUES ('a@x.com','Aman',20), ('b@x.com','Bela',22);
INSERT INTO enrollments VALUES (1,'SQL101','A'), (2,'SQL101','B');
DELETE FROM students WHERE student_id = 1;
SELECT * FROM enrollments;                      -- only Bela's row remains

CREATE TABLE students_backup AS SELECT * FROM students;
```

SQL — 5b: TRUNCATE is refused while a foreign key points at the table

```sql
TRUNCATE TABLE students;
```

ERROR

```
ERROR 1701 (42000): Cannot truncate a table referenced in a foreign key constraint
(`lab`.`enrollments`, CONSTRAINT `fk_enr_student` FOREIGN KEY (`student_id`) REFERENCES
`lab`.`students` (`student_id`))
```

Why: `TRUNCATE` skips row-by-row checks, so MySQL will not run it on a parent table. Use `DELETE FROM students;` (which honors the cascade), or drop the foreign key first.

COMING IN PHASE 3

Phase 3: Querying Basics & Execution Flow. Now that tables are well designed, learn to ask them questions: `SELECT`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, and the hidden order in which MySQL really evaluates them.